

PA 1: Implementing xv6 User Commands and System Calls

Repository: <https://github.com/adeelone/xv6> (branch pa1-syscalls). New files: user/uptime.c, user/time1.c, user/time.c, user/matmul.c, user/sleep.c, kernel/pstat.h. Modified files: Makefile, kernel/proc.h, kernel/proc.c, kernel/trap.c, kernel/defs.h, kernel/syscall.h, kernel/syscall.c, kernel/sysproc.c, user/user.h, user/usys.pl.

Task 1. (5 pts) Implement the uptime utility

I created user/uptime.c to invoke the existing uptime() system call, display the elapsed clock ticks since boot, and exit with status 0. The call flows through the stub in user/usys.S, which puts SYS_uptime into register a7 before triggering the ecall trap into the kernel. Inside kernel/sysproc.c, sys_uptime() acquires tickslock, reads the global ticks variable maintained by the timer interrupt handler, and returns the result. To get it to compile and run inside the xv6 environment, I added \$U/_uptime to UPROGS in the Makefile. I also updated the Makefile to set CPUS to 1, matching the assignment setup.

Figure 1 shows a few test runs from the xv6 shell. The output ticks increase steadily over time (57, 184, 245, 305, 339, 375). The couple of "exec failed" lines were just typos when typing the command at the prompt.



```
,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus
xv6 kernel is booting
init: starting sh
$ uptime
up 57 clock ticks
$ uptime
exec uptime failed
$ uptime
up 184 clock ticks
$ uptime
up 245 clock ticks
$ uptime
up 305 clock ticks
$ uptime
up 339 clock ticks
$ uptime
up 375 clock ticks
$ uptmr
exec uptmr failed
$
```

o direct input to this VM, move the mouse pointer

Figure 1. Running uptime from the xv6 shell

Task 2. (10 pts) Implement a time1 command that reports elapsed time.

In user/time1.c, the program first verifies that the user provided an argument, printing usage information and exiting if missing. It constructs an argument list starting at argv[1] so exec() receives the target command without the leading "time1". The parent measures the starting tick count via uptime(), forks a child process, and lets the child exec() the requested executable. If exec() returns, it means the binary wasn't found, so the child prints an error before exiting. The parent calls wait() until the child completes, grabs the ending uptime(), and prints the difference. Finally, I added \$U/_time1, \$U/_matmul, and \$U/_sleep to UPROGS in the Makefile and placed the test source files (matmul.c and sleep.c) in the user directory.

Figure 2 illustrates the output from testing time1. Running time1 without arguments prints the usage prompt. Running time1 matmul reports 5 ticks, matching matmul's internal measurement. Running two instances concurrently in the background (time1 matmul & time1 matmul &) takes around 11 to 12 ticks per run—roughly double the single-process duration because both processes compete for the same CPU. Running time1 sleep 10 takes 10 ticks since the process sleeps the entire duration. Testing time1 nosuchprogram confirms that exec failures print an error cleanly. My tick totals are slightly lower than the assignment's reference output because QEMU runs faster on my machine, but the proportional relationships match expectations.

A screenshot of a terminal window showing the output of the xv6 kernel booting and running the time1 command. The terminal has a dark background with light-colored text. The output shows the kernel booting, the init process starting a shell, and then several invocations of the time1 command. The first invocation shows usage information. The second invocation shows 5 ticks. The third invocation shows 11 ticks. The fourth invocation shows 12 ticks. The fifth invocation shows 10 ticks. The sixth invocation shows 11 ticks. The seventh invocation shows 10 ticks. The eighth invocation shows 1 tick. The terminal output is as follows:

```
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
ballocc: first 1262 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -gl
,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ time1
usage: time1 command [args...]
$ time1 matmul
Time: 5 ticks
elapsed time: 5 ticks
$ time1 matmul & time1 matmul &
$ Time: 11 ticks
elapsed time: 12 ticks
Time: 10 ticks
elapsed time: 11 ticks

$ time1 sleep 10
elapsed time: 10 ticks
$ time1 nosuchprogram
time1: exec nosuchprogram failed
elapsed time: 1 ticks
$
```

Figure 2. Running time1 from the xv6 shell

Task 3. (10 pts) Keep track of how much cputime a process has used.

To track per-process CPU time, I modified three kernel files:

kernel/proc.h: Added a uint cputime field to struct proc to hold the tick count for each process.

kernel/proc.c: In allocproc(), I initialized p->cputime = 0 right when setting the state to USED. This prevents reused process entries from retaining old CPU counters.

kernel/trap.c: xv6 handles a timer interrupt every tick (which _dev == 2). When an interrupt fires while a process is running, the kernel calls yield(). I added logic to increment the current process's cputime prior to yielding: in usertrap() for user-mode execution (p->cputime++), and in kerneltrap() for kernel-mode execution (myproc()->cputime++). Checking myproc() != 0 ensures we don't increment counters when the CPU is idle.

This approach tallies timer ticks that interrupt an active process. Because timer ticks are discrete, short execution bursts between ticks might go uncounted. Task 4 exposes this metric to user space via wait2(). The changes were committed and pushed to the pa1-syscalls branch.

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

I implemented wait2(int *status, struct rusage *ru), which mimics wait() while populating resource usage information for the exiting child process. The modifications include:

kernel/pstat.h (new): Defined struct rusage containing the cputime field.

user/user.h: Added the struct rusage declaration and wait2() prototype for user code.

user/usys.pl: Added entry("wait2") to generate the assembly stub that sets SYS_wait2 in a7 before issuing ecall.

kernel/syscall.h: Assigned SYS_wait2 to 23 (following sync at 22).

kernel/syscall.c: Registered sys_wait2 in the syscalls[] lookup table.

kernel/sysproc.c: Added sys_wait2(), which retrieves the user-space pointers via argaddr() and invokes kwait2().

kernel/defs.h: Added the kwait2() function declaration.

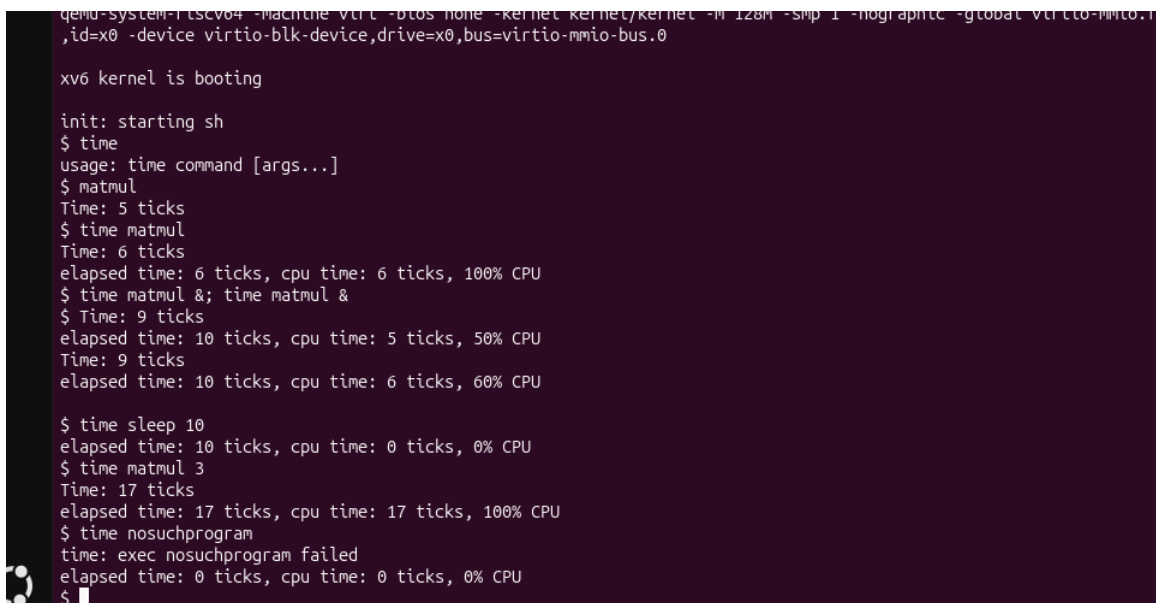
kernel/proc.c: Included pstat.h and implemented kwait2(). When a ZOMBIE child is located, kwait2() copies its cputime into a struct rusage and transfers it to the parent's user memory address using copyout(). This must happen prior to calling freeproc(), which clears the child's proc struct.

Because user processes operate in separate virtual memory spaces, direct pointer dereferencing in the kernel is invalid; `copyout()` handles virtual address translation using the parent's page table. All updates were committed and pushed to `pa1-syscalls`.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

The `user/time.c` program works similarly to `time1`, but replaces `wait()` with `wait2(&status, &ru)` to retrieve `ru.cputime`. The parent captures `uptime()` around the fork-wait sequence to determine total wall time, then calculates %CPU as $\text{cputime} * 100 / \text{elapsed}$. To prevent floating-point or division issues, I checked that `elapsed > 0` and capped the percentage at 100% (since tick rounding can occasionally make `cputime` appear slightly higher than elapsed time). I also added `$U/_time` to `UPROGS` in the Makefile.

Figure 3 demonstrates the functionality. Running `time matmul` on its own yields 6 elapsed ticks and 6 CPU ticks (100% CPU), as expected for a CPU-bound process. Running two instances concurrently yields 10 elapsed ticks with 5 and 6 CPU ticks (50% and 60%), reflecting CPU sharing. Running `time sleep 10` reports 10 elapsed ticks and 0 CPU ticks because sleeping processes don't consume CPU ticks. Running `time matmul 3` scales execution to 17 ticks at 100% CPU. Invoking `time` without arguments outputs usage information, while invalid binary names trigger an exec failure error.



```
qemu-system-i386 -m 128M -smp 1 -hda disk.img -cdrom disk.iso -net nic,model=virtio -net user -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ time
usage: time command [args...]
$ matmul
Time: 5 ticks
$ time matmul
Time: 6 ticks
elapsed time: 6 ticks, cpu time: 6 ticks, 100% CPU
$ time matmul & time matmul &
$ Time: 9 ticks
elapsed time: 10 ticks, cpu time: 5 ticks, 50% CPU
Time: 9 ticks
elapsed time: 10 ticks, cpu time: 6 ticks, 60% CPU

$ time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$ time matmul 3
Time: 17 ticks
elapsed time: 17 ticks, cpu time: 17 ticks, 100% CPU
$ time nosuchprogram
time: exec nosuchprogram failed
elapsed time: 0 ticks, cpu time: 0 ticks, 0% CPU
$
```

Figure 3. Running time from the xv6 shell

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

This assignment provided a complete walkthrough of system calls in xv6: from `usys.pl` code generation, register setup, `ecall` traps, system call table lookup, argument extraction with `argaddr()`, to returning data via `copyout()`. Implementing `wait2()` highlighted how multiple kernel files depend on coordinated changes. Working with tick-based time accounting demonstrated how timer interrupts sample process activity, making the difference between CPU time and wall-clock time clear when running concurrent workloads.

I encountered a few specific challenges during development. Modern xv6-riscv function names differed from older references (e.g., `kwait()` instead of `wait()`, `pause()` instead of `sleep()`, and an updated signature for `copyout()`), which required checking the codebase directly. A debug print leftover from a previous exercise was messing up timing counts, so I guarded it with a Makefile compile flag (`TRACE=1`). I also hit a minor bug where an unsaved edit to `kernel/proc.h` caused a compiler error regarding `cputime` missing from `struct proc`. Lastly, GitHub authentication required switching from account passwords to a personal access token, and transient network drops in my VM were resolved by committing locally first before pushing.

Extra Credit (5 points). Discuss limitations of our time command.

Child processes are not counted. `wait2()` only collects statistics from the direct child process. If that child spawns sub-children, their CPU usage isn't added to the total. If the child exits before its own children finish, those sub-processes get adopted by `init` and their stats are lost to time. (Linux avoids this by accumulating child stats upon `wait` calls).

Tick granularity. Sampling CPU time via timer interrupts introduces measurement variance. Short tasks might complete between tick interrupts without getting charged, or a process might get charged a full tick despite only running briefly before the interrupt fires. This explains minor split discrepancies like 5 vs 6 ticks during concurrent execution and necessitates clamping percentage to 100%.

No user/system split. Unlike standard Linux time utilities, user-space and kernel-space execution times are combined into a single total rather than broken down into user/sys components.

Elapsed time includes overhead. Wall-clock measurement starts before `fork()` and ends after `wait2()` returns, incorporating parent setup, exec overhead, and scheduling latencies rather than strictly isolating child execution duration.

Integer math. %CPU uses integer division, truncating fractional values rather than rounding.

Only runs a single program. Direct execution via `exec()` prevents support for shell constructs like pipes, redirections, or shell built-ins (e.g., `cd`).

Timings are printed even on failure. Failed `exec` attempts still print a summary row showing 0 ticks, and exit status codes are not reported to the user.